Python Exercises - Upskilling (5.0)

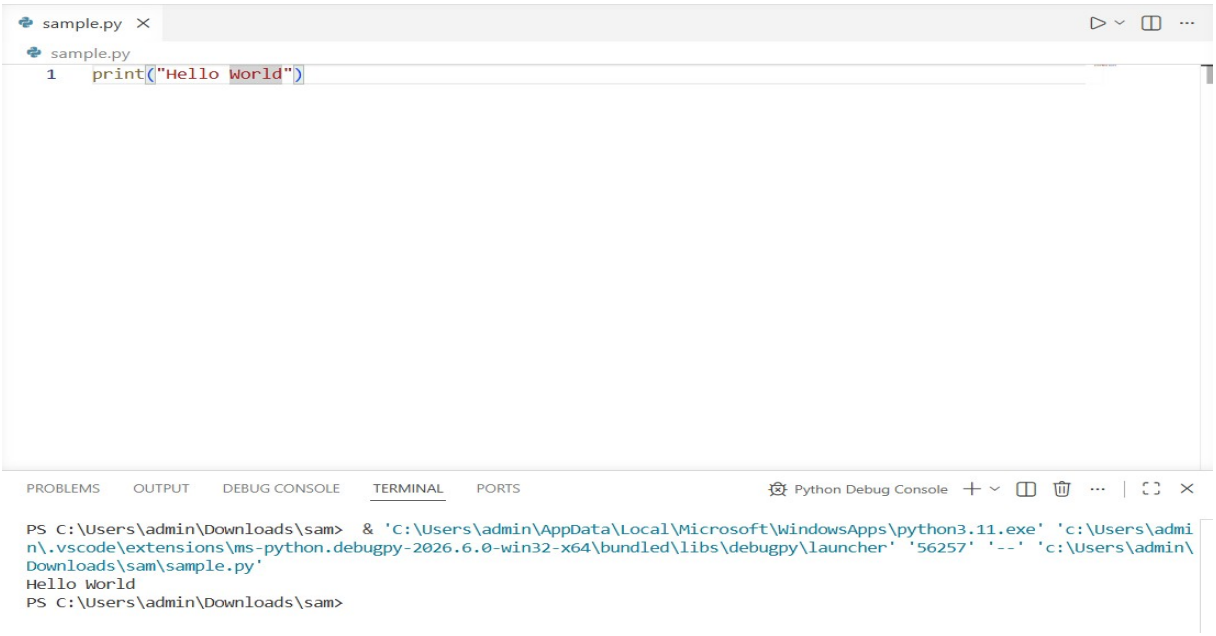
Name: Balaji A

SupersetId: 7995004

Basic concepts hands-on questions

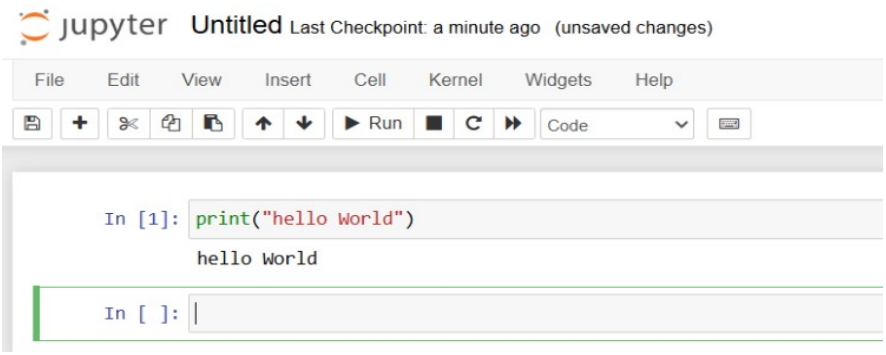
Installing Python s Running Programs

1. **Simple Hello World**
   - **Objective:** Create first Python file.
   - **Task:** Write and execute hello program.
   - **Instructions:**
     - Create first.py with print("Hello World!").
     - Run ***python first.py***.
     - Share terminal output.

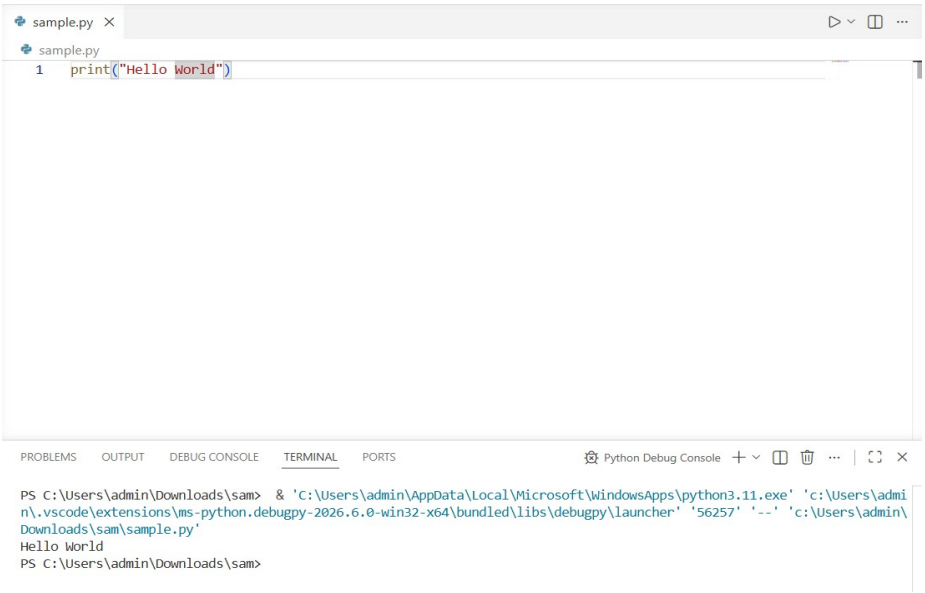


Text Editors and IDEs

2. **Jupyter Notebook**
   - **Objective:** Create Jupyter notebook.
   - **Task:** Basic notebook operations.
   - **Instructions:**
     - Run ***pip install notebook***, then ***jupyter notebook***.
     - Create new notebook, add code/math cells.



3. **VS Code Setup**
   - **Objective:** Configure Python in VS Code.
   - **Task:** Install Python extension.
   - **Instructions:**
     - Install VS Code Python extension.
     - Create .py file, verify IntelliSense.
     - Run with Ctrl+F5.



Data Types

4. **Float Precision**
   - **Objective:** Handle floating-point arithmetic with functions + validation + formatted output.
   - **Task:** Calculate net salary after tax and print with 2 decimals.
   - **Instructions:**
     - salary = 75000.5, tax_rate = 0.18.
     - Use a function
     - Validate salary C tax rate
     - Use variables
     - Format output to 2 decimal places

In [7]:
```
def func(salary,tax_rate):
    print(f"Net Salary: {(salary-(salary*tax_rate)):.2f}")
salary=float(input("Enter Salary"))
tax_rate=float(input("Enter Tax Rate"))
func(salary,tax_rate)
```

```
Enter Salary75000.5
Enter Tax Rate0.18
Net Salary: 61500.41
```

Variables

5. **Multiple Assignment**
   - **Objective:** Use multiple assignment with basic validation + function + formatted output.
   - **Task:** Unpack (x, y) coordinates and display them.
   - **Instructions:**
     - Use a function
     - Validate coordinates
     - Use multiple assignment
     - Print coordinates

```
In [9]: def display(x, y):
            if x >= 0 and y >= 0:
                print("Coordinates:", x, y)
            else:
                print("Invalid coordinates")

        x, y = 10, 20

        display(x, y)
```

```
Coordinates: 10 20
```

Mathematical Operations

6. **Modulo Operator**
   - **Objective:** Use the modulo operator with function + validation + clean output.
   - **Task:** Check if a number is even or odd using %.
   - **Instructions:**
     - Use a function
     - Validate number
     - Compute remainder
     - Print "Even" or "Odd"
     - Use variable number = 17

```
In [11]: num=int(input("Enter Number to Check Even or Odd:"))
         def func(num):
             if(num%2==0):
                 print("Even")
             else :
                 print("Odd")
         func(num)
```

```
Enter Number to Check Even or Odd:17
Odd
```

7. **Floor Division**
   - **Objective:** Perform integer division with function + validation + formatted output.
   - **Task:** Split a total bill equally among people.
   - **Instructions:**
     - Use a function
     - Validate bill C number of people
     - Use floor division //

- Print individual share
- Variables: total_bill = 1250, people = 4

```
13]: def split_bill(total_bill, people):
         if total_bill <= 0 or people <= 0:
             print("Invalid Input")
         else:
             share = total_bill // people
             print("Individual Share:", share)

     total_bill = int(input("Enter Total Bill:"))
     people = int(input("No of People:"))

     split_bill(total_bill, people)
```

```
Enter Total Bill:1250
No of People:4
Individual Share: 312
```

8. **Min/Max Functions**

- **Objective:** Use built-in min() and max() with validation + function + structured output.
- **Task:** Find the highest and lowest salary in a list.
- **Instructions:**
  - Use a function
  - Validate the salary list
  - Compute min C max
  - Print the results
  - Use list: [50000, 75000, 62000, 95000]

```
In [14]: def find_salary(salaries):
             print("Highest Salary:", max(salaries))
             print("Lowest Salary:", min(salaries))

         salaries = [50000, 75000, 62000, 95000]

         find_salary(salaries)
```

```
Highest Salary: 95000
Lowest Salary: 50000
```

User Input

9. **Basic Input**

- **Objective:** Read and validate user input using a function + formatted output.
- **Task:** Get the user's name and greet them.
- **Instructions:**
  - Use a function

- Validate empty input
- Use input()
- Print greeting message

```
In [16]: str=input("Enter Name:")
         def greet(str):
             print("Welcome",str)
         greet(str)
```

```
Enter Name:Gokul
Welcome Gokul
```

---

10. Numeric Input

- **Objective:** Convert user input to a number with validation + function + formatted output.
- **Task:** Ask user for age, validate, convert to integer, and print next year's age.
- **Instructions:**
  - Use a function
  - Validate numeric input
  - Convert using int()
  - Print "Next year you'll be <age+1>"

```
In [17]: age=int(input("Enter Current Age:"))
         def next_year(age):
             print("Next year you'll be ",(age+1))
         next_year(age)
```

```
Enter Current Age:21
Next year you'll be  22
```

11. Float Input

- **Objective:** Handle decimal input with validation + function + formatted output.
- **Task:** Convert weight from kilograms to pounds.
- **Instructions:**
  - Use a function
  - Validate decimal input
  - Convert kg → lbs using "lbs = kg * 2.20462"
  - Print the weight in pounds.

```
In [17]: age=int(input("Enter Current Age:"))
         def next_year(age):
             print("Next year you'll be ",(age+1))
         next_year(age)
```

```
Enter Current Age:21
Next year you'll be  22
```

Control Flow

12. Simple If

- **Objective:** Basic conditional with function, validation, and user interaction.
- **Task:** Check if a student passes based on marks.
- **Instructions:**

- Use a function
- Validate marks
- Use a variable (marks = 75)
- Print Pass/Fail

```
In [21]: mark=int(input("Enter Mark:"))
         if(mark>35):
             print("Pass")
         else:
             print("Better Luck Next Time")

         Enter Mark:40
         Pass
```

13. **If-Else**

- **Objective:** Two-way decision using functions + validation + formatted output.
- **Task:** Check if a number is even or odd.
- **Instructions:**
  - Use a function
  - Validate input
  - Use variable num = 8
  - Print "Even" or "Odd"

```
In [21]: mark=int(input("Enter Mark:"))
         if(mark>35):
             print("Pass")
         else:
             print("Better Luck Next Time")

         Enter Mark:40
         Pass
```

14. **If-Elif-Else**

- **Objective:** Multi-way decision using functions + validation + structured output.
- **Task:** Assign a grade based on the score.
- **Instructions:**
  - Use a function
  - Validate score
  - Use variable score = 88

- Print grade A/B/C
- Add slight enhancements over the basic version

```
In [2]: mark=int(input())
        if(mark>0 and mark<100):
            if(mark>80):
                print("A")
            elif(mark<=80 and mark>60):
                print("B")
            else:
                print("C")
```

```
88
A
```

15. For Loop Basics

- **Objective:** Iterate a fixed number of times with a function + validation + formatted output.
- **Task:** Print numbers from 1 to 5 using range().
- **Instructions:**
  - Use a function
  - Validate loop count
  - Use for i in range(5)
  - Print numbers 1–5

```
n=int(input("Enter Loop Count:"))
def func(n):
    if(n<=0):
        print("Loop count must be positive")
    for i in range(1,n+1):
        print(i)
func(n)
```

```
Enter Loop Count:5
1
2
3
4
5
```

16. While Loop

- **Objective:** Use a condition-based loop with validation + function + formatted output.
- **Task:** Countdown from 5 to 1 using a while loop.
- **Instructions:**
  - Use a function
  - Validate count value
  - Use while count > 0:
  - Print count each step
  - Decrease using count -= 1

```
count=int(input("Enter Count:"))
def func(count):
    while count>0:
        print(count)
        count=count-1
func(count)
```

```
Enter Count:5
5
4
3
2
1
```

17. **Break Statement**

- **Objective:** Use a break statement to exit a loop early with function + validation + clean output.
- **Task:** Find and print the first even number in a given range.
- **Instructions:**
  - Use a function
  - Validate the range size
  - Loop with for i in range(...)
  - Use modulo % to check even
  - Break once the first even is found

```
n=int(input("Enter Start:"))
m=int(input("Enter End:"))
def func(n,m):
    if n>0 and m>=n and m>0:
        for i in range(n,m+1):
            if(i%2==0):
                print(i)
                break
    else:
        print("Invalid range")
func(n,m)
```

```
Enter Start:5
Enter End:10
6
```

18. **Continue Statement**

- **Objective:** Use continue to skip even numbers, with a function + validation + formatted output.
- **Task:** Sum only the odd numbers in a given range.
- **Instructions:**

- Use a function
- Validate the range
- Loop with range(10)
- Skip even numbers using continue
- Accumulate only odd numbers

```
n=int(input("Enter End:"))
def func(n):
    if(n>=0):
        total=0
        for i in range(n):
            if(i%2==0):
                continue
            total+=i
        print("Sum of Odd Nums:",total)
func(n)
```

```
Enter End:10
Sum of Odd Nums: 25
```

19. **Pass Statement**

- **Objective:** Use pass as a placeholder inside a function, with simple structure and formatted output.
- **Task:** Define an empty function using pass and print confirmation.
- **Instructions:**
  - Use a function
  - Add placeholder logic with pass
  - Call the function
  - Print "Function defined"

```
def func():
    pass
func()
print("Function defined")
```

```
Function defined
```

Whitespace Importance

20. **Consistent Indentation**

- **Objective:** Demonstrate clean, consistent indentation using a nested if inside a function.
- **Task:** Print "Nested" when both conditions are True.
- **Instructions:**
  - Use a function
  - Apply exactly 4 spaces per indentation level
  - Use nested if-statements
  - Print "Nested"
  - Print confirmation message

```python
def check_nested(condition1, condition2):
    if condition1:
        if condition2:
            print("Nested")
            print("Both conditions are True")

check_nested(True, True)
```

```
Nested
Both conditions are True
```

Organizing Programs

21. **Comment Usage**
    - **Objective:** Show proper documentation using meaningful comments + a function.
    - **Task:** Add explanatory comments and print the total salary.
    - **Instructions:**
        - Use clear comments
        - Use a function
        - Define base and bonus
        - Calculate total
        - Print result

```python
def calculate_salary(base_salary, bonus):
    total_salary = base_salary + bonus
    print("Total Salary:", total_salary)

base_salary = 50000
bonus = 10000

calculate_salary(base_salary, bonus)
```

```
Total Salary: 60000
```

Modules

22. **Import Standard Module**
    - **Objective:** Use the math module inside a function with validation + formatted output.
    - **Task:** Calculate the area of a circle using math.pi and a given radius.
    - **Instructions:**
        - Import math
        - Use a function
        - Validate radius

```python
import math

def calculate_area(radius):

    if radius < 0:
        print("Invalid radius")
        return

    area = math.pi * radius ** 2

    print(f"Area of Circle: {area:.2f}")

radius = 5

calculate_area(radius)
```

```
Area of Circle: 78.54
```

- Use formula: area = math.pi × radius²
- Print area with 2 decimals

23. **All Import**

- **Objective:** Use from module import * along with simple utility usage inside a function.
- **Task:** Import everything from a standard module and use a few functions.
- **Instructions:**
  - Use from math import *
  - Use a function
  - Validate some input
  - Demonstrate usage (e.g., sqrt, pow, pi)
  - Print results

```
from math import *

def math_operations(number):

    if number < 0:
        print("Invalid input")
        return

    print(f"Square Root: {sqrt(number)}")
    print(f"Power: {pow(number, 2)}")
    print(f"Value of Pi: {pi}")


math_operations(16)
```

```
Square Root: 4.0
Power: 256.0
Value of Pi: 3.141592653589793
```

Functions

24. **Parameters**

- **Objective:** Pass arguments into a function with validation + structured output.
- **Task:** Create a function that adds two numbers and prints the result.
- **Instructions:**
  - Use a function with parameters
  - Validate inputs
  - Return the sum
  - Print the result of add(5, 3)

```
def add(num1, num2):
    if not isinstance(num1, (int, float)) or not isinstance(num2, (int, float)):
        print("Error: Both inputs must be numerical values.")
        return None

    return num1 + num2


result = add(5, 3)
print(result)
```

```
8
```

25. **Multiple Parameters**

- **Objective:** Handle multiple arguments in a function with validation + formatted output.
- **Task:** Calculate the area of a rectangle using length and width.
- **Instructions:**
  - Use a function that accepts two arguments
  - Validate the numbers
  - Calculate and return area
  - Print area(5, 3)

```
def calculate_area(length, width):

    if length <= 0 or width <= 0:
        return "Invalid dimensions"

    area = length * width

    return area


print(calculate_area(5, 3))
```

```
15
```

Built-in Functions

26. **Len Function**

- **Objective:** Use len() inside a function with basic validation + structured output.
- **Task:** Get the length of a given string.
- **Instructions:**
  - Use a function
  - Validate the string
  - Use len(text)
  - Print the length

```
def get_string_length(text):
    if not isinstance(text, str):
        print("Error: Input must be a string.")
        return None

    length = len(text)
    print(f"The length of the string is: {length}")
    return length


get_string_length("Hello, World!")
```

```
The length of the string is: 13
13
```

File I/O

27. **Write to File**
    - **Objective:** Create and write to a text file using a function + clean structure.
    - **Task:** Save a greeting message into a text file.
    - **Instructions:**
        - Use a function
        - Open a file in write mode
        - Write "Hello World" into the file
        - Print confirmation message

```
def write_message():

    file = open("greeting.txt", "w")

    file.write("Hello World")

    file.close()

    print("Message written to file")


write_message()
```

```
Message written to file
```

28. **Read from File**
    - **Objective:** Read text from a file using a function with basic error handling.
    - **Task:** Open a text file and display its contents.
    - **Instructions:**
        - Use a function
        - Open file in read mode
        - Read full content using read()
        - Print file content
        - Handle missing file gracefully

```
def read_file_content(file_path):
    try:
        with open(file_path, "r") as file:
            content = file.read()
            print(content)
            return content
    except FileNotFoundError:
        print(f"Error: The file '{file_path}' does not exist.")
        return None


read_file_content("sample.txt")
```

```
Error: The file 'sample.txt' does not exist.
```

Error Handling

29. **Basic Try-Except**
    - **Objective:** Catch simple runtime errors using try-except with a function and clear output.
    - **Task:** Safely divide two numbers and handle division-by-zero errors.
    - **Instructions:**
        - Use a function
        - Use try-except
        - Handle ZeroDivisionError
        - Print result or error message

```python
def divide_numbers(a, b):

    try:
        result = a / b
        print(f"Result: {result}")

    except ZeroDivisionError:
        print("Error: Cannot divide by zero")


divide_numbers(10, 2)
divide_numbers(10, 0)
```

```
Result: 5.0
Error: Cannot divide by zero
```

Lists

30. Create List

- **Objective:** Initialize a list with basic validation + a function + clean output.
- **Task:** Create a shopping cart list and display the items.
- **Instructions:**
  - Use a function
  - Validate list items
  - Initialize list with values [100, 250, 75]
  - Print the cart contents

```python
def display_shopping_cart(cart_items):
    if not isinstance(cart_items, list) or len(cart_items) == 0:
        print("Error: The shopping cart must be a non-empty list.")
        return

    print("Shopping Cart Contents:")
    for item in cart_items:
        print(f"- Item Value: {item}")


shopping_cart = [100, 250, 75]
display_shopping_cart(shopping_cart)
```

```
Shopping Cart Contents:
- Item Value: 100
- Item Value: 250
- Item Value: 75
```

31. Append to List

- **Objective:** Add a single item to a list using a function with validation.
- **Task:** Add a new expense to an existing expenses list.
- **Instructions:**
  - Use a function
  - Validate the expense amount

- Use .append()
- Print the updated expenses list

```
def add_expense(expenses, amount):

    if amount <= 0:
        print("Invalid expense amount")
        return

    expenses.append(amount)

    print("Updated Expenses List:", expenses)


expenses = [200, 500, 1000]

add_expense(expenses, 300)
```

```
Updated Expenses List: [200, 500, 1000, 300]
```

Dictionaries

32. Update Dictionary

- **Objective:** Perform bulk dictionary updates using a function with basic validation.
- **Task:** Merge employee details from two dictionaries.
- **Instructions:**
  - Use a function
  - Validate dictionary inputs
  - Use .update()
  - Print the updated employee data

```
def merge_employee_details(base_details, new_details):
    if not isinstance(base_details, dict) or not isinstance(new_details, dict):
        print("Error: Both inputs must be dictionaries.")
        return None

    base_details.update(new_details)
    print("Updated Employee Data:", base_details)
    return base_details


employee_base = {"id": 101, "name": "Alice", "role": "Developer"}
employee_update = {"role": "Senior Developer", "department": "Engineering"}

merge_employee_details(employee_base, employee_update)
```

```
Updated Employee Data: {'id': 101, 'name': 'Alice', 'role': 'Senior Developer', 'department': 'Engineering'}
{'id': 101,
 'name': 'Alice',
 'role': 'Senior Developer',
 'department': 'Engineering'}
```

33. Nested Dictionary

- **Objective:** Work with nested dictionaries using a function and basic validation.
- **Task:** Store department-wise employee data and retrieve a specific employee's salary.
- **Instructions:**
  - Use a nested dictionary
  - Use a function to fetch data
  - Validate department and employee name
  - Print the employee salary

```python
def get_salary(data, department, employee):

    if department not in data:
        print("Invalid department")
        return

    if employee not in data[department]:
        print("Employee not found")
        return

    print(f"Salary of {employee}: {data[department][employee]}")


employees = {
    "HR": {
        "Arun": 50000,
        "Priya": 60000
    },
    "IT": {
        "Rahul": 75000,
        "Sneha": 80000
    }
}

get_salary(employees, "IT", "Rahul")
```

```
Salary of Rahul: 75000
```

Tuples and Sets

34. Create Tuple

- **Objective:** Use an immutable tuple with a function and basic validation.
- **Task:** Store fixed coordinates and display them.
- **Instructions:**
  - Use a tuple
  - Use a function
  - Validate coordinate values
  - Print coordinates in a readable format

```python
def process_coordinates(coords):
    if not isinstance(coords, tuple) or len(coords) != 2:
        print("Error: Input must be a tuple containing exactly 2 elements (X, Y).")
        return None

    if not all(isinstance(val, (int, float)) for val in coords):
        print("Error: Coordinate values must be numbers.")
        return None

    x, y = coords
    print(f"Fixed Coordinates -> X: {x}, Y: {y}")
    return coords


location_coords = (13.0827, 80.2707)
process_coordinates(location_coords)
```

```
Fixed Coordinates -> X: 13.0827, Y: 80.2707
(13.0827, 80.2707)
```

35. Set Intersection

- **Objective:** Find common elements between two sets using a function and basic validation.
- **Task:** Identify common skills between two skill sets.
- **Instructions:**
  - Use sets
  - Use a function
  - Validate inputs

- Find intersection using C
- Print common skills

```python
def find_common_skills(set1, set2):

    if not isinstance(set1, set) or not isinstance(set2, set):
        print("Invalid input")
        return

    common_skills = set1 & set2

    print("Common Skills:", common_skills)


skills1 = {"Python", "Java", "SQL"}
skills2 = {"Python", "C++", "SQL"}

find_common_skills(skills1, skills2)
```

```
Common Skills: {'Python', 'SQL'}
```

OOP concepts

Classes

36. Multiple Instances

- **Objective:** Create multiple objects from a class and access their attributes.
- **Task:** Create an employee roster using multiple instances of a class.
- **Instructions:**
  - Define a class with __init_
  - Create multiple objects (instances)
  - Add a method to display employee info
  - Print employee names

```python
class Employee:
    def __init__(self, name, role):
        self.name = name
        self.role = role

    def display_info(self):
        print(f"Employee Name: {self.name} | Role: {self.role}")


emp1 = Employee("Alice", "Software Engineer")
emp2 = Employee("Bob", "Product Manager")
emp3 = Employee("Charlie", "Data Analyst")

emp1.display_info()
emp2.display_info()
emp3.display_info()
```

```
Employee Name: Alice | Role: Software Engineer
Employee Name: Bob | Role: Product Manager
Employee Name: Charlie | Role: Data Analyst
```

37. Method Chaining

- **Objective:** Demonstrate method chaining by returning self from methods.
- **Task:** Create an employee object, set salary, apply raise, and display final salary.
- **Instructions:**
  - Create a class
  - Methods should return self
  - Use method chaining
  - Add basic validation
  - Print final salary

```python
class Employee:
    def __init__(self):
        self.salary=0
    def set_salary(self,s):
        if s>0:
            self.salary=s
        return self
    def add_raise(self,r):
        if r >0:
            self.salary=self.salary+r
        return self
    def display(self):
        print("Final Salary:",self.salary)
e=Employee()
e.set_salary(20000).add_raise(5000).display()
```

```
Final Salary: 25000
```

Custom Classes s Inheritance

38. **Polymorphism**
    - **Objective:** Demonstrate polymorphism using method overriding.
    - **Task:** Show different behaviors of the same method (work()) for different employee types.
    - **Instructions:**
        - Create a base class with a common method
        - Override the method in child classes
        - Store objects in a list
        - Call the same method for all objects
        - Observe different outputs

```python
class Employee:
    def work(self):
        print("Employee is performing general duties.")


class Developer(Employee):
    def work(self):
        print("Developer is writing and debugging code.")


class Manager(Employee):
    def work(self):
        print("Manager is organizing meetings and planning timelines.")


employees = [Employee(), Developer(), Manager()]

for emp in employees:
    emp.work()
```

```
Employee is performing general duties.
Developer is writing and debugging code.
Manager is organizing meetings and planning timelines.
```

39. **Class Methods**
    - **Objective:** Use a class method as a factory to create objects from a formatted string.
    - **Task:** Create an Employee object from a string like "Shubh,75000".
    - **Instructions:**
        - Use @classmethod
        - Parse string input

- Convert salary to integer
- Return a new object using cls
- Print employee details

```python
class Employee:

    def __init__(self, name, salary):
        self.name = name
        self.salary = salary

    @classmethod
    def from_string(cls, data):

        name, salary = data.split(",")

        return cls(name, int(salary))

    def display(self):

        print(f"Name: {self.name}")
        print(f"Salary: {self.salary}")


employee = Employee.from_string("Shubh,75000")

employee.display()
```

```
Name: Shubh
Salary: 75000
```

Real-time simulation hands-on questions

40. Employee Management System

- **Objective:** Use classes, dictionaries, lists, and JSON file I/O together.
- **Task:** Create an employee management system that can save and load employee data.
- **Instructions:**
  - Create Employee class with __init__and __str_
  - Store employees in a dictionary
  - Save data to emps.json
  - Load data back from file
  - Print all employees

```
import json


class Employee:
    def __init__(self, emp_id, name, role):
        self.emp_id = emp_id
        self.name = name
        self.role = role

    def __str__(self):
        return f"ID: {self.emp_id} | Name: {self.name} | Role: {self.role}"


def save_employees(employee_dict, filename="emps.json"):
    serializable_dict = {
        emp_id: {"name": emp.name, "role": emp.role}
        for emp_id, emp in employee_dict.items()
    }
    with open(filename, "w") as file:
        json.dump(serializable_dict, file, indent=4)


def load_employees(filename="emps.json"):
    try:
        with open(filename, "r") as file:
            data = json.load(file)
            return {
                emp_id: Employee(emp_id, info["name"], info["role"])
                for emp_id, info in data.items()
            }
    except FileNotFoundError:
        return {}


employee_roster = {
    "101": Employee("101", "Alice", "Developer"),
    "102": Employee("102", "Bob", "Manager"),
}

save_employees(employee_roster)

loaded_roster = load_employees()

for employee in loaded_roster.values():
    print(employee)
```

```
ID: 101 | Name: Alice | Role: Developer
ID: 102 | Name: Bob | Role: Manager
```

41. Data Analysis Pipeline

- **Objective:** Use modules, functions, lists, file handling, and error handling together.
- **Task:** Process sales data from a file and calculate statistics.
- **Instructions:**
  - Import statistics module
  - Read numeric data from sales.txt
  - Handle file and data errors using try-except
  - Calculate mean and median
  - Print a statistics summary

```python
import statistics

def analyze_sales():

    try:
        file = open("sales.txt", "r")

        sales = []

        for line in file:
            sales.append(float(line.strip()))

        file.close()

        mean_value = statistics.mean(sales)
        median_value = statistics.median(sales)

        print("Sales Statistics")
        print(f"Mean: {mean_value}")
        print(f"Median: {median_value}")

    except FileNotFoundError:
        print("File not found")

    except ValueError:
        print("Invalid data in file")


analyze_sales()
```

```
Sales Statistics
Mean: 1800.0
Median: 1800.0
```

42. **Configuration Manager**

- **Objective:** Use classes, inheritance, dictionaries, and file I/O to manage application configuration.
- **Task:** Create a configurable system that loads database settings from a file.
- **Instructions:**
  - Create a base Config class
  - Create DatabaseConfig class inheriting from Config
  - Use configparser module
  - Load settings from db.ini
  - Validate required keys

```python
import configparser
import os


class Config:
    def __init__(self, filepath):
        self.filepath = filepath
        self.parser = configparser.ConfigParser()

    def load(self):
        if not os.path.exists(self.filepath):
            print(f"Error: Configuration file '{self.filepath}' not found.")
            return False
        self.parser.read(self.filepath)
        return True


class DatabaseConfig(Config):
    def __init__(self, filepath):
        super().__init__(filepath)
        self.settings = {}

    def load_and_validate(self):
        if not self.load():
            return None

        if "database" not in self.parser.sections():
            print("Error: Missing [database] section in configuration file.")
            return None

        db_section = self.parser["database"]
        required_keys = ["host", "port", "user", "password", "dbname"]
        validated_settings = {}

        for key in required_keys:
            if key not in db_section or db_section[key].strip() == "":
                print(f"Error: Missing or blank required key '{key}'.")
                return None
            validated_settings[key] = db_section[key]

        self.settings = validated_settings
        print("Database settings successfully loaded and validated.")
        return self.settings


with open("db.ini", "w") as f:
    f.write("[database]\n")
    f.write("host = localhost\n")
    f.write("port = 5432\n")
    f.write("user = admin\n")
    f.write("password = secret\n")
    f.write("dbname = production_db\n")

db_config = DatabaseConfig("db.ini")
config_data = db_config.load_and_validate()
if config_data:
    print(config_data)
```

```
Database settings successfully loaded and validated.
{'host': 'localhost', 'port': '5432', 'user': 'admin', 'password': 'secret', 'dbname': 'production_d
b'}
```

43. CSV Data Processor

- **Objective:** Use file I/O, lists, dictionaries, and comprehensions to analyze CSV data.
- **Task:** Analyze employee salary data from a CSV file.
- **Instructions:**
  - Read employees.csv using the csv module
  - Convert rows into a list of dictionaries
  - Filter employees with salary > 50000 using list comprehension
  - Calculate and print average salary

```python
import csv

def process_csv():

    employees = []

    with open("employees.csv", "r") as file:

        reader = csv.DictReader(file)

        for row in reader:
            row["salary"] = int(row["salary"])
            employees.append(row)

    high_salary = [emp for emp in employees if emp["salary"] > 50000]

    average_salary = sum(emp["salary"] for emp in employees) / len(employees)

    print("Employees with salary > 50000")

    for emp in high_salary:
        print(emp)

    print(f"Average Salary: {average_salary}")


process_csv()
```

```
Employees with salary > 50000
{'name': 'Priya', 'salary': 60000}
{'name': 'Rahul', 'salary': 75000}
Average Salary: 57500.0
```

44. Expense Tracker

- **Objective:** Use file handling, date processing, lists, dictionaries, and comprehensions for expense analysis.
- **Task:** Analyze current month expenses and calculate totals per category.
- **Instructions:**

- Read expenses.csv (date, amount, category)
- Filter current month using datetime
- Group expenses by category using dictionary
- Calculate total amount per category
- Print summary

```python
import csv
from datetime import datetime


def analyze_expenses(file_path):
    current_date = datetime.now()
    current_year = current_date.year
    current_month = current_date.month

    category_totals = {}

    try:
        with open(file_path, "r") as file:
            reader = csv.DictReader(file)

            for row in reader:
                try:
                    expense_date = datetime.strptime(
                        row["date"].strip(), "%Y-%m-%d"
                    )
                except ValueError:
                    continue

                if (
                    expense_date.year == current_year
                    and expense_date.month == current_month
                ):
                    category = row["category"].strip()
                    try:
                        amount = float(row["amount"].strip())
                    except ValueError:
                        continue

                    category_totals[category] = (
                        category_totals.get(category, 0.0) + amount
                    )

    except FileNotFoundError:
        print(f"Error: The file '{file_path}' was not found.")
        return None

    print(
        f"--- Expense Summary for {current_date.strftime('%B %Y')} ---"
    )
    for category, total in category_totals.items():
        print(f"{category}: ${total:.2f}")

    return category_totals


with open("expenses.csv", "w", newline="") as f:
    writer = csv.writer(f)
    writer.writerow(["date", "amount", "category"])
    writer.writerow(["2026-05-15", "45.50", "Food"])
    writer.writerow(["2026-05-20", "12.00", "Transport"])
    writer.writerow(["2026-05-25", "120.00", "Food"])
    writer.writerow(["2026-04-10", "85.00", "Utilities"])

analyze_expenses("expenses.csv")
```

```
--- Expense Summary for May 2026 ---
Food: $165.50
Transport: $12.00
{'Food': 165.5, 'Transport': 12.0}
```

45. API Response Handler

- **Objective:** Use requests, JSON handling, classes, and error handling to process API responses.
- **Task:** Fetch weather data from an API and display temperature and conditions safely.
- **Instructions:**
  - Install C use requests
  - Fetch weather data (JSON)
  - Parse temperature and conditions
  - Handle 404 and network errors
  - Format output neatly

```python
import requests

class WeatherAPI:

    def get_weather(self, city):

        url = f"https://wttr.in/{city}?format=j1"

        try:
            response = requests.get(url)

            if response.status_code == 404:
                print("City not found")
                return

            response.raise_for_status()

            data = response.json()

            temperature = data["current_condition"][0]["temp_C"]
            condition = data["current_condition"][0]["weatherDesc"][0]["value"]

            print(f"City: {city}")
            print(f"Temperature: {temperature}°C")
            print(f"Condition: {condition}")

        except requests.exceptions.RequestException:
            print("Network error occurred")

        except KeyError:
            print("Invalid data received")


weather = WeatherAPI()

weather.get_weather("Chennai")
```

```
City: Chennai
Temperature: 33°C
Condition: Clear
```

Additional real-time simulation hands-on questions

46. Complete Calculator Program

- **Objective:** Combine input, functions, error handling.
- **Task:** Build a robust calculator that supports + , - , * , /.
- **Instructions:**
  - Create function calculate(a, b, op) handling +, -, *, /.
  - Use input() for numbers/operation.
  - Handle ZeroDivisionError and ValueError.
  - Print formatted result.

```python
def calculate(a, b, op):
    try:
        if op == "+":
            return a + b

        elif op == "-":
            return a - b

        elif op == "*":
            return a * b

        elif op == "/":
            return a / b

        else:
            return "Invalid Operator"

    except ZeroDivisionError:
        return "Cannot divide by zero"

try:
    a = float(input("Enter first number: "))
    b = float(input("Enter second number: "))
    op = input("Enter operator: ")

    result = calculate(a, b, op)

    print("Result:", result)

except ValueError:
    print("Invalid input")
```

```
Enter first number:  23
Enter second number:  40
Enter operator:  +
Result: 63.0
```

47. Shopping Cart System

- **Objective:** Use classes, lists, control flow, and the math module to build a simple shopping cart system.
- **Task:** Perform complete cart operations and print a receipt with GST.
- **Instructions:**
  - Create CartItem class with price and quantity
  - Create ShoppingCart class with:
    - add item
    - remove item
    - calculate total
  - Apply 18% GST
  - Print final receipt

```python
class CartItem:
    def __init__(self, name, price, quantity):
        self.name = name
        self.price = price
        self.quantity = quantity

class ShoppingCart:
    def __init__(self):
        self.items = []

    def add_item(self, item):
        self.items.append(item)

    def calculate_total(self):
        total = sum(item.price * item.quantity for item in self.items)
        gst = total * 0.18
        return total + gst

    def print_receipt(self):
        for item in self.items:
            print(item.name, item.quantity, item.price)

        print("Final Total:", self.calculate_total())

cart = ShoppingCart()

cart.add_item(CartItem("Laptop", 50000, 1))
cart.add_item(CartItem("Mouse", 500, 2))

cart.print_receipt()
```

```
Laptop 1 50000
Mouse 2 500
Final Total: 60180.0
```

48. Temperature Converter GUI

- **Objective:** Use classes, user input, modules, and formatted output.
- **Task:** Convert temperatures between Celsius, Fahrenheit, and Kelvin.
- **Instructions:**
  - Converter class

- Convert C ↔ F ↔ K
- Use input() menu
- Format output to 2 decimals
- Use tabulate module for table display

```
from tabulate import tabulate

class Converter:
    def c_to_f(self, c):
        return (c * 9/5) + 32

    def f_to_c(self, f):
        return (f - 32) * 5/9

converter = Converter()

celsius = float(input("Enter Celsius: "))

fahrenheit = converter.c_to_f(celsius)

table = [["Celsius", celsius], ["Fahrenheit", round(fahrenheit, 2)]]

print(tabulate(table, headers=["Type", "Value"]))
```

```
Enter Celsius:  39
Type          Value
----------  -------
Celsius        39
Fahrenheit    102.2
```

49. Backup Utility

- **Objective:** Use file operations, modules, sets, and error handling to build a smart backup tool.
- **Task:** Copy files to a backup folder while skipping duplicates and logging operations.
- **Instructions:**
  - Use shutil module.
  - Copy files but skip duplicates using sets.
  - Handle FileNotFoundError, PermissionError.
  - Log operations to backup.log.

```
import shutil
import os

def backup_files(files, backup_folder):

    copied_files = set()

    try:

        if not os.path.exists(backup_folder):
            os.mkdir(backup_folder)

        log_file = open("backup.log", "a")

        for file in files:

            if file in copied_files:
                log_file.write(f"Skipped duplicate: {file}\n")
                continue

            shutil.copy(file, backup_folder)

            copied_files.add(file)

            log_file.write(f"Copied: {file}\n")

        log_file.close()

        print("Backup completed")

    except FileNotFoundError:
        print("File not found")

    except PermissionError:
        print("Permission denied")


file_list = ["sales.txt", "employees.csv", "sales.txt"]

backup_files(file_list, "backup")
```

```
Backup completed
```

50. URL Shortener

- **Objective:** Use dictionaries, hashing, classes, and modules to build a simple URL shortening service.
- **Task:** Create a basic URL shortener that can shorten URLs and retrieve the original URL.

- **Instructions:**
  - Import hashlib
  - URLShortener class with dictionary storage
  - Generate 6-character hash from URL
  - Lookup (redirect simulation) functionality

```
import hashlib
class URLShortener:
    def __init__(self):
        self.urls = {}
    def shorten_url(self, original_url):
        short_code = hashlib.md5(original_url.encode()).hexdigest()[:6]
        self.urls[short_code] = original_url
        return short_code
    def get_original_url(self, short_code):
        if short_code in self.urls:
            return self.urls[short_code]
        return "URL not found"
shortener = URLShortener()
url = "https://www.google.com"
short_code = shortener.shorten_url(url)
print("Short Code:", short_code)
print("Original URL:", shortener.get_original_url(short_code))
```

```
Short Code: 8ffdef
Original URL: https://www.google.com
```

51. Gradebook System

- **Objective:** Use dictionaries, functions, control flow, and file I/O for student grade management.
- **Task:** Manage student grades, calculate GPA, save/load data, and compute class average.
- **Instructions:**
  - Dictionary: student → list of grades
  - Functions: add grade, calculate GPA, save/load
  - Handle invalid grade inputs
  - Print class average

```
import json

students = {}

def add_grade(name, grade):
    if 0 <= grade <= 100:
        students.setdefault(name, []).append(grade)

def calculate_gpa(grades):
    return sum(grades) / len(grades)

add_grade("Vishal", 90)
add_grade("Vishal", 85)

with open("grades.json", "w") as file:
    json.dump(students, file)

for name, grades in students.items():
    print(name, "GPA:", calculate_gpa(grades))

class_average = sum(
    calculate_gpa(g) for g in students.values()
) / len(students)

print("Class Average:", class_average)
```

```
Vishal GPA: 87.5
Class Average: 87.5
```

52. Task Scheduler

- **Objective:** Use classes, the datetime module, lists, and sorting to manage tasks.
- **Task:** Create a task scheduler that prioritizes tasks by due date and identifies overdue tasks.
- **Instructions:**
  - Task class with due_date and priority
  - Store tasks in a list
  - Sort tasks by due date using datetime
  - Filter overdue tasks
  - Print sorted task schedule

```python
from datetime import datetime

class Task:
    def __init__(self, name, due_date, priority):
        self.name = name
        self.due_date = datetime.strptime(due_date, "%Y-%m-%d")
        self.priority = priority

tasks = [
    Task("Project", "2026-06-10", 1),
    Task("Assignment", "2026-05-20", 2)
]

tasks.sort(key=lambda x: x.due_date)

today = datetime.now()

print("Task Schedule")

for task in tasks:
    status = "Overdue" if task.due_date < today else "Pending"

    print(task.name, task.due_date.date(), status)
```

```
Task Schedule
Assignment 2026-05-20 Overdue
Project 2026-06-10 Pending
```

53. Inventory Manager

- **Objective:** Use classes, inheritance, dictionaries, and sets to manage warehouse inventory.

- **Task:** Track products, stock levels, and generate low-stock alerts.
- **Instructions:**
  - Product base class
  - Perishable and Electronics inherit from Product
  - Track stock levels using a dictionary
  - Use sets for low-stock alerts
  - Print inventory summary

```python
class Product:
    def __init__(self, name, stock):
        self.name = name
        self.stock = stock

class Perishable(Product):
    pass

class Electronics(Product):
    pass

inventory = {
    "Milk": Perishable("Milk", 5),
    "Laptop": Electronics("Laptop", 20)
}

low_stock = {
    name for name, product in inventory.items()
    if product.stock < 10
}

print("Inventory Summary")

for name, product in inventory.items():
    print(name, "-", product.stock)

print("Low Stock Alerts:", low_stock)
```

```
Inventory Summary
Milk - 5
Laptop - 20
Low Stock Alerts: {'Milk'}
```

54. Budget Planner

- **Objective:** Use classes, control flow, external modules, and visualization to track a monthly budget.
- **Task:** Create a monthly budget planner that tracks spending, alerts overspending, and visualizes data.
- **Instructions:**
  - Category class with limit and spent
  - Input expenses using a loop
  - Use matplotlib for pie chart visualization
  - Alert when category exceeds budget

```python
import matplotlib.pyplot as plt

class Category:
    def __init__(self, name, limit):
        self.name = name
        self.limit = limit
        self.spent = 0

    def add_expense(self, amount):
        self.spent += amount

        if self.spent > self.limit:
            print(f"{self.name} exceeded budget!")

food = Category("Food", 5000)
travel = Category("Travel", 3000)

food.add_expense(4500)
food.add_expense(1000)

travel.add_expense(2000)

labels = [food.name, travel.name]
values = [food.spent, travel.spent]

plt.pie(values, labels=labels, autopct="%1.1f%%")

plt.title("Monthly Budget")

plt.show()
```

```
Food exceeded budget!
```